

Building a RAG demo takes an afternoon.

Building one that stays correct when a deploy fails halfway through is **a different engineering problem.**

THE SYSTEM

Multi-tenant AI knowledge platform

Cited answers over an institution's own documents

THE FOCUS

Reliability & evidence

Durability, isolation, observability, cost

THE CONSTRAINT

Free-tier infrastructure

512 MB, one worker, no managed queue

THE CLAIM

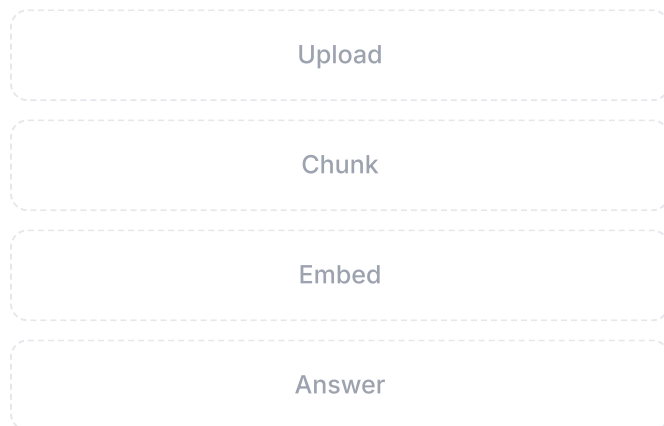
Production-oriented

Not production-ready — and I'll explain why

Where most RAG projects stop

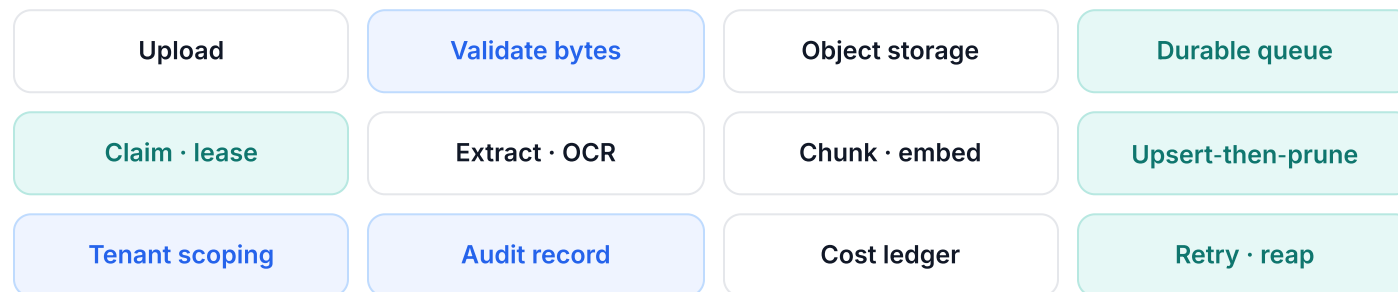
The demo working is the beginning of the engineering, not the end of it.

TYPICAL TUTORIAL



Every box works on the happy path. None of them says what happens when the process dies between two of them.

THE SAME FEATURE, BUILT TO SURVIVE

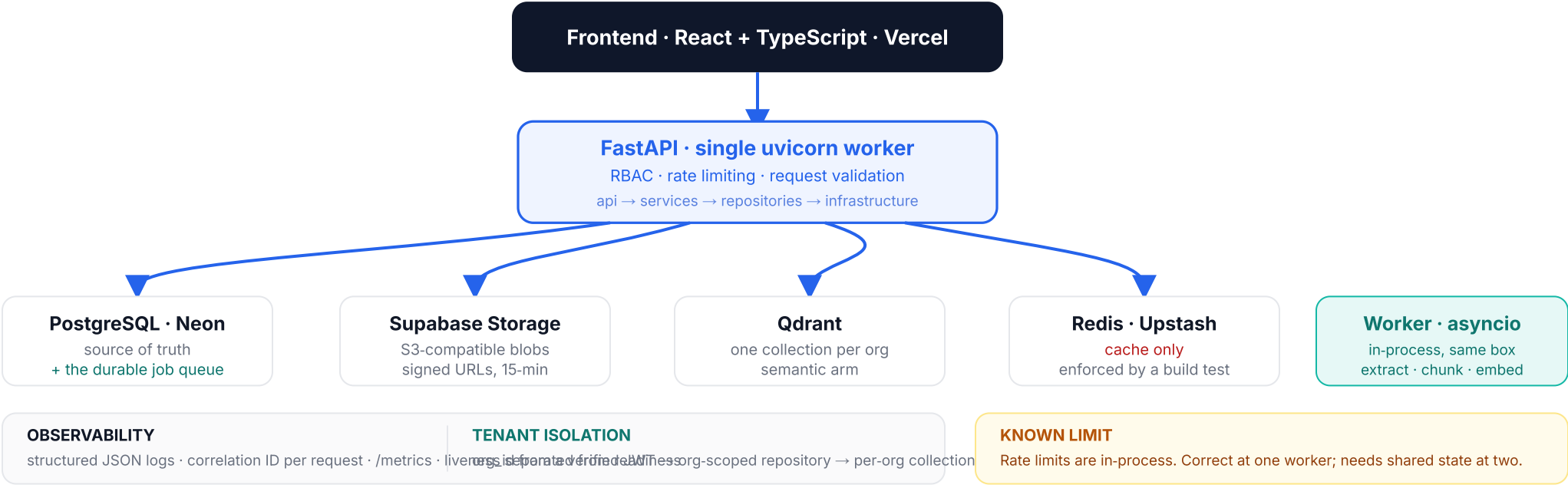


■ security & isolation ■ durability & recovery

The upload endpoint returns **202 Accepted** in well under a second and the work continues without it. That single change is what forces almost every box above to exist.

One deployment graph

Each store holds exactly one kind of responsibility.



The decision I'd defend hardest is one where I removed a feature

⊗ WHAT I DIDN'T SHIP

A confidence badge

🛡️ High Confidence

It demos beautifully. Every RAG tutorial ships one.

But a label means choosing cutoffs — above 0.7 is “High”, below 0.4 is “Low”. I had **no labelled dataset** to validate a single one of those numbers against.

It would have been a number I invented, rendered so it looks measured. And once it is on screen, people quote it back at you.

✅ WHAT I SHIPPED INSTEAD

The measurements, with their basis

Documents cited	3
Excerpts used	9 of 14 retrieved
Closest match	0.71
Refusal floor	0.35

The score is **never shown without the threshold it was compared against** — 0.71 means nothing on its own. Cited and retrieved counts appear together, because an answer using 2 of 14 passages is a different claim from one using 9 of 14.

All of it was already being computed and written to a trace table. It was simply **discarded from the response**.

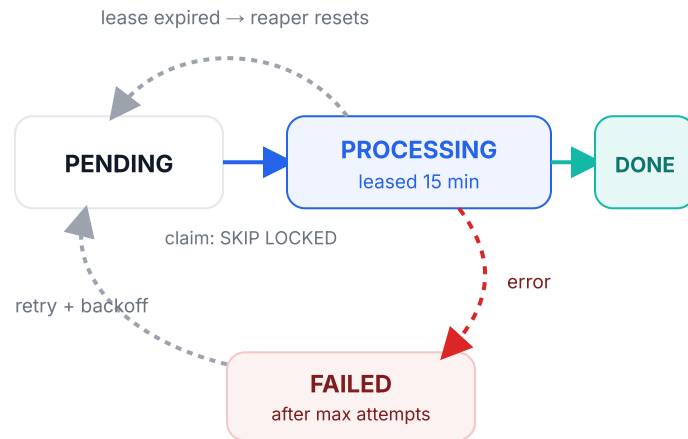
Every decision names what it gave up

Five choices, each with the alternative I considered and rejected.

01	PROBLEM A dropped ingestion job is silent data loss, not a slow request.	CHOSEN Postgres job queue — FOR UPDATE SKIP LOCKED, lease reaper, jittered backoff.	REJECTED Redis or a managed queue.	TRADE-OFF Slower than Redis, but no second stateful system to operate for durability I already had.
02	PROBLEM Nothing stops a future contributor using the cache as a queue.	CHOSEN Redis scoped to caching , with a test that fails the build on any other use.	REJECTED A convention in a README.	TRADE-OFF A test that checks source, not behaviour — unusual, but conventions rot and tests don't.
03	PROBLEM Cosine similarity and Postgres ts_rank are not comparable scales.	CHOSEN Fuse on rank position (RRF), keeping the semantic score separately.	REJECTED Summing or weighting raw scores.	TRADE-OFF Loses score magnitude — but the refusal threshold must read the semantic score, not the fused one.
04	PROBLEM Long ingestion must leave the request path; the tier has no worker service.	CHOSEN In-process asyncio worker over the durable table.	REJECTED A separate worker deployment.	TRADE-OFF Shares a fate with the API. Named as a constraint, with a written migration trigger.
05	PROBLEM An unaudited change and a missing cost row are not the same severity.	CHOSEN Opposite contracts : audit fails closed in the same transaction; cost fails open .	REJECTED One uniform logging policy.	TRADE-OFF Two behaviours to reason about, so a logging row can never break an answer.

Designed for recovery, not for luck

A system is judged by how it behaves when it breaks.



FAILURE

Instance sleeps after 15 minutes idle

RECOVERY

The queue is a table, so it survives. The worker re-claims on wake.

FAILURE

Worker dies mid-ingestion

RECOVERY

Row stays leased → reaper resets it after the lease expires → bounded retry

FAILURE

Crash midway through re-indexing

RECOVERY

New vectors are written **before** old ones are deleted, so a crash degrades rather than destroys

FAILURE

Two workers claim the same job

RECOVERY

Impossible by construction, and there is an integration test that proves it

FAILURE

Client disconnects mid-answer

RECOVERY

The partial answer is still recorded — an abandoned request must not vanish from the log

Six hours of silence

The outage had nothing to do with the AI. Neither did my first fix.

SYMPTOM

TLS handshake completed. Request fully sent. **Zero bytes returned.** Not a slow cold start — three separate attempts up to 90 seconds returned nothing at all.

RULED OUT

DNS resolved correctly. The same sandbox reached other hosts in under a second. So: the edge accepted the connection and the origin never answered.

ROOT CAUSE

A migration created a Postgres enum type **explicitly**, then `create_table` created the same type **again**. It could never have succeeded on any database. The first deploy got far enough to leave the type behind, so every boot afterwards hit the identical wall — and the start script runs migrations before the server.

```
psycopg2.errors.DuplicateObject: type "ingestion_job_status" already exists
```

MY FIRST FIX

Also wrong. I set `create_type=False` on a generic `sa.Enum` — a parameter that only exists on the dialect-specific type and is **silently ignored** there. Same error, one line further down.

REAL FIX

The correct type, plus `checkfirst` so it tolerates the orphaned type already in production. This time I **compiled the DDL and counted the statements** before claiming it worked: two `CREATE TYPE` before, one after.

LESSON

Verifying beats reasoning — **especially about your own fix.** The CI step that should have caught this existed. Nobody had checked it was passing.

The failure that taught me most was in a migration, not a model

What I measure — and what I refuse to

The second column is the harder engineering decision.

👁 MEASURED, AND ACTED ON

Refusal rate

The one quality signal computable with no labels at all. A rising rate means retrieval is degrading.

Uncited-answer rate

Answers that cite nothing are the shape a hallucination takes here.

Token cost per answer

Attributed to org, user and document. Real provider usage, not an estimate.

Retrieval traces

The query that actually ran, the chunks returned with their scores, and what was cited — stored per answer.

Latency percentiles

Retrieval, generation, and time-to-first-token measured separately.

⚠ DELIBERATELY NOT COMPUTED

Recall & precision

Both need a labelled golden set. There isn't one, so the columns don't exist — and two tests assert they never appear.

Groundedness / hallucination rate

Same reason. A number nothing computed is worse than a blank, because a blank gets questioned.

A confidence label

Would require thresholds nothing validated. The API returns the score and the floor instead.

“We benchmarked the reranker”

I evaluated a cross-encoder and declined to ship it — but on **a handful of manual queries, not a golden set**. Directionally right, not rigorous. Recorded that way, with three revisit triggers.

What the demo never teaches

Building the AI feature was the easy part.

Reliability is **harder than generation**, and less visible.

A good decision names the alternative it rejected.

The best feature I shipped this month was **one I deleted**.

Verify, don't reason — especially about your own fix.

A system that admits what it doesn't know is more useful than one that sounds certain.

CampusBrain

A production-oriented AI knowledge platform — and an argument about engineering honesty.



WHAT IT IS

A multi-tenant knowledge platform answering questions over an institution's own documents, with inline citations and an explicit refusal when nothing in the corpus is close enough.



WHAT I LEARNED

The operating layer *is* the product. Durability, isolation, cost attribution and honest uncertainty took far longer than the retrieval pipeline, and they're what make it trustworthy.



WHY IT EXISTS

To show I can design, operate and debug a system under real constraints — and know the difference between what I've measured and what I've merely assumed.

I'd value critique from backend, platform and AI engineers — particularly on the queue choice and the refusal threshold.

Those are the two decisions I'm least certain about. Every trade-off, including the reversed ones, is written down.